Prediction

- The Math of Prediction
- Intra-Prediction
- Inter-Prediction

Predictive Coding: Why Residual Matter

- **Predictive Coding**
  - Encode residual, not signal → $e(n) = x(n) - \hat{x}(n)$
  - Why don't we encode pixels directly?
    - Pixels: large dynamic range → high entropy
    - Residuals: mostly near 0 → lower entropy
    - Entropy coding: compression gain comes from reduced variance, and skewed distribution
- **DPCM (Differential Pulse Code Modulation)**

$$\hat{x}(n) = x(n-1) \text{ , } \rightarrow \quad e(n) = x(n) - x(n-1)$$

  - E.g., Original : [10, 11, 12, 12, 13] → Residual: [10, 1, 1, 0, 1]
    → Small dynamic range

Predictive Coding

- **Why Prediction Works**
  - Natural image = highly correlated → Neighbor pixels are similar
- Example (next slide)
  - **Left:** Original image and its histogram
    - Pixel values (0-255) are uncorrelated across image but highly correlated locally.
    - The overall distribution is broad.
    - High Entropy: Many bits are needed to represent each pixel directly.
  - **Right**: Prediction Residual and its histogram
    - Use DPCM: left-neighbor prediction (no prediction for left-most pixels)
    - The distribution becomes a sharp Laplacian (or Gaussian) peak centered at 0.
    - Low Entropy: efficient for Variable Length Coding (Huffman/ Arithmetic Coding).

DPCM prediction

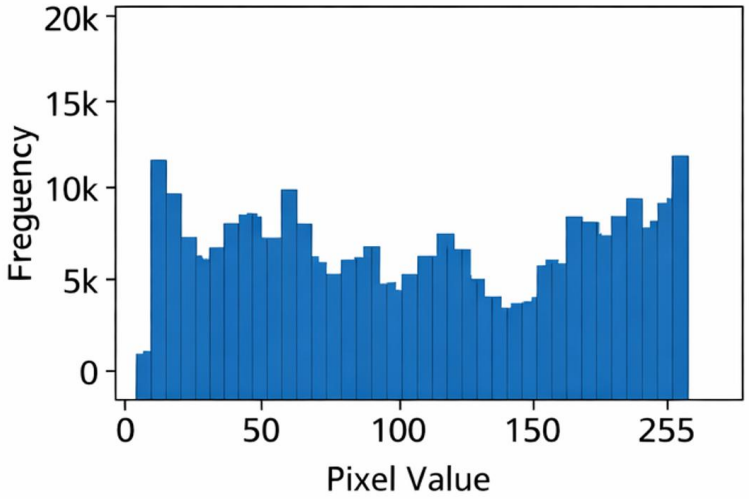
Original Image



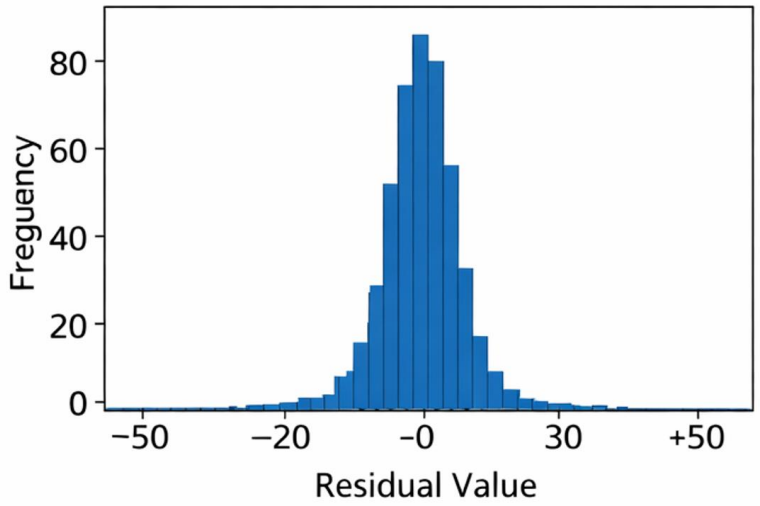
Residual Image



Histogram of Original Pixels



Histogram of Prediction Residuals



Even with a very simple predictor (just copying the left pixel), the residual distribution becomes highly concentrated around zero.

DPCM (Differential Pulse Coding Module)

- **DPCM** is the fundamental framework for predictive coding, whether it's in space (Intra) or time (Inter). It formalizes the concept of encoding the difference.
- **The Encoder**
  - **Prediction**: Generate a prediction $\hat{x}(n)$ for the current sample $x(n)$ based on previous encoded and decoded samples.
  - **Residual Calculation**

    $$e(n) = x(n) - \hat{x}(n)$$, where $e(n)$: prediction error (residual)
  - Quantization & Coding: $e(n)$ is quantized and then entropy coded.
- **The Decoder**
  - Entropy Decoding & Inverse Quantization: Retrieve the quantized residual
  - **Reconstruction**: $\tilde{x}(n) = \hat{x}(n) + \tilde{e}(n)$
  - Notice that $\tilde{x}(n)$ Is not exactly $x(n)$ due to quantization error.

Predictive Coding: The "Closed-loop" Requirement

- **In PCM**
  - $e(n) = x(n) - \hat{x}(n)$
  - $\hat{x}(n) = f(\tilde{x}(n-1))$, $\tilde{x}$: reconstructed data (In DPCM: $\hat{x}(n) = \tilde{x}(n-1)$)

→ the prediction $\hat{x}(n)$ is a function of *reconstructed pixels*

→ **Closed-loop** prediction

- **Can we predict from original pixels?**
  - The problem: **Open-loop** Prediction
  - Predicting from the *original pixels* causes the encoder and decoder to lose synchronization because decoder doesn't have original pixels
    - **Drift occurs**→ Error accumulates over time → catastrophic degradation

Open-Loop vs Closed-Loop Prediction

Setup:

- Predictor: left neighbor
- First pixel is sent directly
- Quantization of residual: q = round(e / 4), dequantized residual = 4q

Original 4×4 block X

$$X = \begin{bmatrix} 50 & 52 & 54 & 56 \\ 60 & 62 & 64 & 66 \\ 70 & 72 & 74 & 76 \\ 80 & 82 & 84 & 86 \end{bmatrix}$$

	Open-loop predict from ORIGINAL left pixel				**Closed-loop** predict from RECONSTRUCTED left pixel			
	x_1	x_2	x_3	x_4	x_1	x_2	x_3	x_4
Original row	50	52	54	56	50	52	54	56
	orig x_1 = 50	orig x_2 = 52	orig x_3 = 54					
Reconstructed row used for prediction					50	54	54	58
					recon $\tilde{x}_1$ = 50	recon $\tilde{x}_2$ = 54	recon $\tilde{x}_3$ = 54	
Predictor	—	50	52	54	—	50	54	54
Residual e	—	$e_2 = 2$	$e_3 = 2$	$e_4 = 2$	—	$e_2 = 2$	$e_3 = 0$	$e_4 = 2$
Quantized q	—	$q_2 = 1$	$q_3 = 1$	$q_4 = 1$	—	$q_2 = 1$	$q_3 = 0$	$q_4 = 1$
Dequantized 4q	—	$4q_2 = 4$	$4q_3 = 4$	$4q_4 = 4$	—	$4q_2 = 4$	$4q_3 = 0$	$4q_4 = 4$
Reconstructed $\tilde{x}$	50	$\tilde{x}_2 = 54$	$\tilde{x}_3 = 58$	$\tilde{x}_4 = 62$	50	$\tilde{x}_2 = 54$	$\tilde{x}_3 = 54$	$\tilde{x}_4 = 58$
	Reconstructed row: [50, 54, 58, 62]				**Reconstructed row: [50, 54, 54, 58]**			

Open-loop mismatch:
encoder uses original references,
but decoder only has reconstructed references.

Closed-loop match:
encoder and decoder use
the same reconstructed references.

Key idea: real codecs use closed-loop prediction to avoid mismatch / drift.

Open-loop vs. Closed-loop

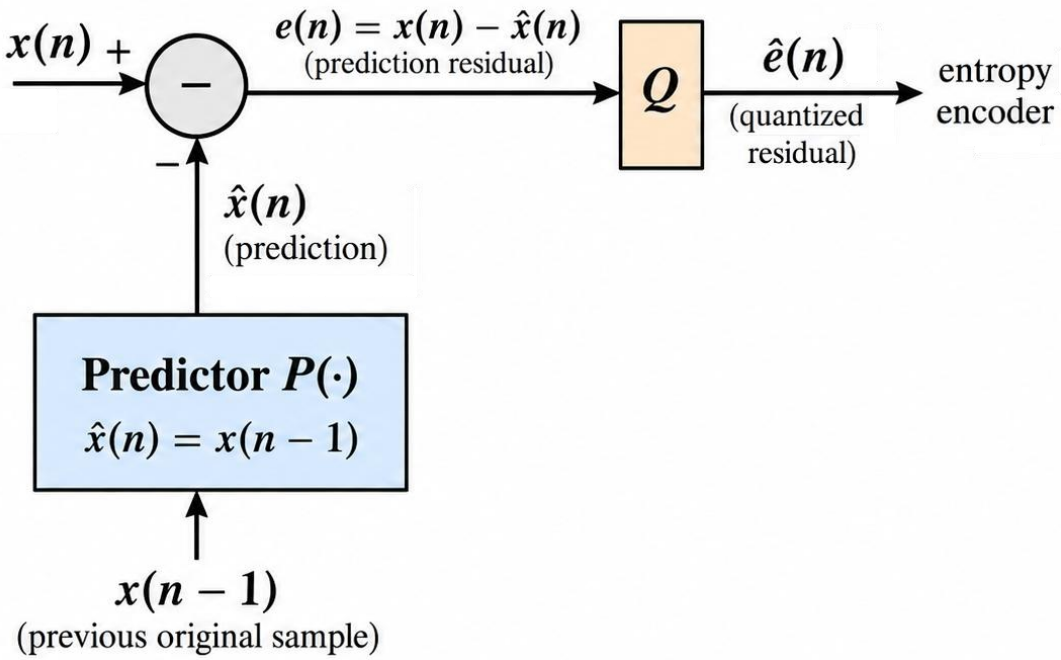
- **Open-Loop Prediction**
  - Encoder predicts from **original pixels**: $\hat{\boldsymbol{x}}_{\boldsymbol{enc}} = f(x_{original_past})$
  - Decoder can only predict from **decoded pixels**: $\hat{\boldsymbol{x}}_{\boldsymbol{dec}} = f(x_{decoded_past})$
  - Because $x_{original} \neq x_{decoded}$ (due to quantization) → $\hat{\boldsymbol{x}}_{\boldsymbol{enc}} \neq \hat{\boldsymbol{x}}_{\boldsymbol{dec}}$
  - **Drift Error**: Error accumulates over time → catastrophic degradation.
- **Close-Loop Prediction**
  - The Encoder must contain a mini-Decoder.
  - Both Encoder and Decoder predict from the **reconstructed (decoded) pixels**:

$$\hat{\boldsymbol{x}}_{\boldsymbol{enc}} = f(x_{original_past}) \quad \rightarrow \quad \hat{\boldsymbol{x}}_{\boldsymbol{enc}} = \hat{\boldsymbol{x}}_{\boldsymbol{dec}}$$

  - This ensures and the only error in the final image is the local quantization error of the current block, which does not propagate.

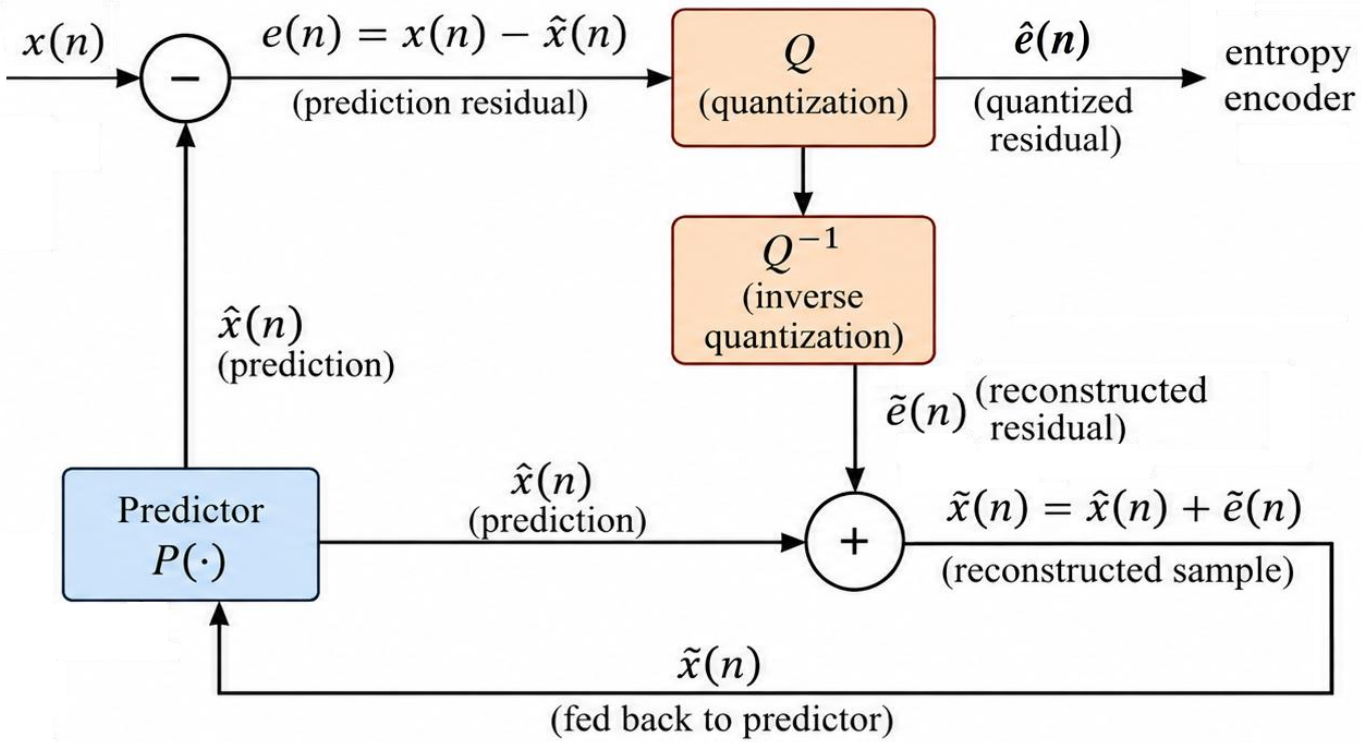
Open-Loop vs. Closed-Loop Diagram

Open-loop Prediction



- The encoder predicts the current signal based on the original past signal $x(n-1)$.

Closed-loop Prediction



- The encoder includes a “mini decoder“ .
- The prediction is based on reconstructed past signal $\tilde{x}(n-1)$.

Open-Loop vs. Closed-Loop

Feature	Open-Loop Prediction	Closed-Loop (DPCM)
Reference Source	**Original** signal $x(n-1)$	**Reconstructed** signal $\tilde{x}(n-1)$
Encoder Complexity	**Low** (Simple feed-forward)	**High** (Includes feedback loop)
Synchronization	**Loss of Sync:** Encoder and Decoder predictions diverge.	**Perfect Sync:** Both sides use the identical reference.
Error Propagation	**High (Drift Error):** Quantization errors accumulate over time.	**Zero Drift:** Errors are contained within the current sample.

Prediction

- The Math of Prediction
- Intra-Prediction
- Inter-Prediction

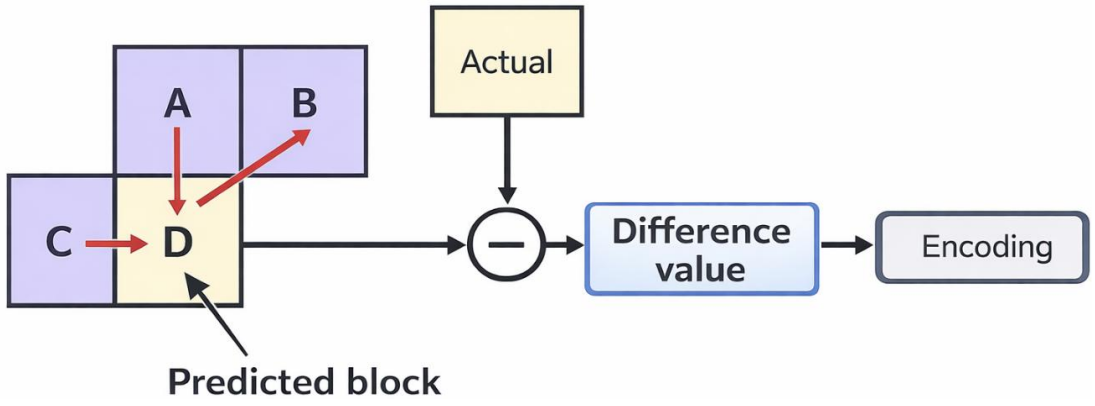
From 1D to 3D Prediction — A Dimensional Perspective

- **1D Prediction: Basic DPCM**
  - **Concept:** Prediction along a single sequence or a scan line.
  - **Source:** Uses the immediate previous sample to predict the current sample
  - **Redundancy:** Exploits Serial Correlation (typical in audio signals or 1D sensors).
- **2D Prediction: Intra-prediction**
  - **Concept:** Prediction within a spatial plane (the current frame).
  - **Source:** Uses reconstructed pixels from the top and left boundaries of current block.
  - **Redundancy:** Exploits Spatial Correlation (neighboring pixels in an image are similar).
- **3D Prediction: Inter-prediction**
  - **Concept:** Prediction across the temporal axis (Time).
  - **Source:** Uses pixels from previous or future frames (Reference Frames).
  - **Redundancy:** Exploits Temporal Correlation (objects in a video don't change much between frames, they just move).

Concept of Spatial Redundancy

- **Spatial Redundancy: Why Intra-prediction Works?**
  - **Observation**: Natural images are spatial correlated. Nearby pixels are similar.
    - Residual becomes: small, concentrated around 0, easier to compress
  - **Intra-prediction Defined**: A technique to predict the current block using only information from the same frame.
  - **The Goal**: To reduce the energy of the signal. Instead of encoding the actual pixel values, we only encode the Residual (Error).
  - **Scan Order Constraint**

    We can only use pixels from the Top and Left as references because the decoder processes blocks from top-left to bottom-right.

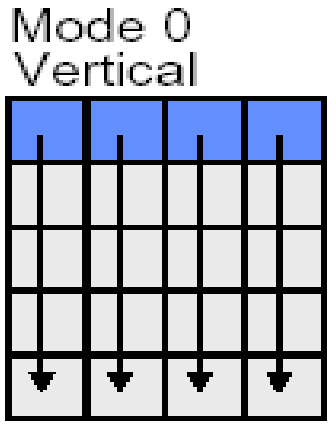


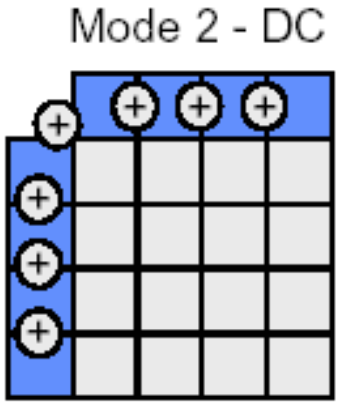
Intra-Prediction: Basic Mode

- **Prediction Unit (**Ex. 4x4**)**
  - A= Above reference
  - L = Left reference
  - ? = pixels to predict

A A A A A
L ? ? ? ?
L ? ? ? ?
L ? ? ? ?
L ? ? ? ?

- **Basic Modes (Common Modes)**
  - Mode 0: Vertical
    - $\hat{x}(i,j) = \text{Above(j)}$ // copy from top
  - Mode 1: Horizontal
    - $\hat{x}(i,j) = \text{Left(i)}$ // copy from left
  - Mode 2: DC
    - $\hat{x}(i,j) =$ average of all top and left samples





Intra-Prediction: Basic Mode

- Example: 4x4 block

M	A	B	C	D	E	F	G	H
I	a	b	c	d				
J	e	f	g	h				
K	i	j	k	l				
L	m	n	o	p				

Mode 0: Vertical prediction mode

$$\begin{array}{cccccccc} a & = & e & = & i & = & m & = & A \\ b & = & f & = & j & = & n & = & B \\ c & = & g & = & k & = & o & = & C \\ d & = & h & = & l & = & p & = & D \end{array}$$

Mode 1: Horizontal prediction mode

$$\begin{array}{cccccccc} a & = & b & = & c & = & d & = & I \\ e & = & f & = & g & = & h & = & J \\ i & = & j & = & k & = & l & = & K \\ m & = & n & = & o & = & p & = & L \end{array}$$

Mode 2: DC prediction

If A, B, C, D, I, J, K and L are available, then

$$P = \frac{A+B+C+D+I+J+K+L+4}{8}$$

If only A, B, C and D exist, then

$$P = \frac{A+B+C+D+2}{4}$$ (ex: the top row)

if only I, J, K, and L exist, then

$$P = \frac{I+J+K+L+2}{4}$$ (ex:leftmost col)

If all the samples (A~D, I~L) are not available, then

P = (1 << ($BitDepth_Y$-1))

$BitDepth_Y$ is bit length of Luma Samples。

(ex:left-top corner)

Intra-Prediction: Angular Prediction

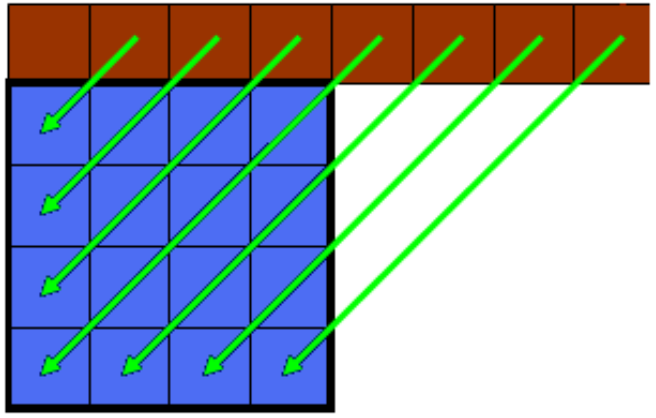
- **Angular Modes**
  - Predict along **directional angles**
  - Use interpolation
  - Handles diagonal textures: Different textures (like a sloping edge) require different projection angles to minimize the residual.

	M	A	B	C	D	E	F	G	H
I		a	b	c	d				
J		e	f	g	h				
K		i	j	k	l				
L		m	n	o	p				

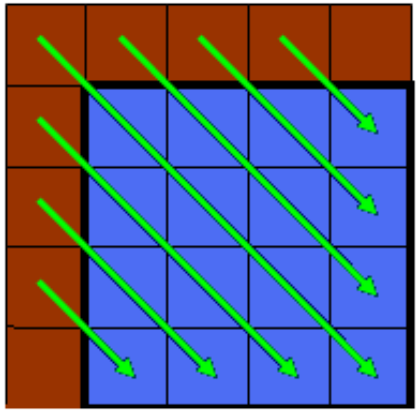
Mode 3

$$
\begin{aligned}
a &= \frac{A+2B+C+2}{4} \\
b=e &= \frac{B+2C+D+2}{4} \\
c=f=i &= \frac{C+2D+E+2}{4} \\
d=g=j=m &= \frac{D+2E+F+2}{4} \\
h=k=n &= \frac{E+2F+G+2}{4} \\
l=o &= \frac{F+2G+H+2}{4} \\
p &= \frac{G+3H+2}{4}
\end{aligned}
$$

Mode 3
Diagonal Down Left

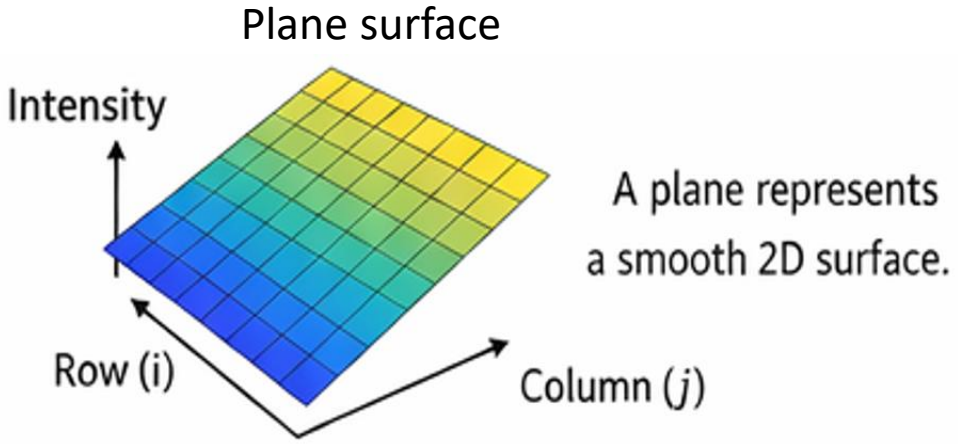


Mode 4
Diagonal Down Right



Intra-Prediction: Plane Prediction

- **Plane Prediction**
  - To predict a block that has a smooth color gradient rather than a flat, solid color.
  - **The Problem**: Vertical, Horizontal, and DC modes often create "blocking artifacts" in smooth areas because they cannot represent a gradual change in brightness.
  - **The Solution**: Use a **Linear Surface Model** to fit the boundary pixels.
    - It can model both horizontal and vertical changes simultaneously, forming a smooth gradient.
- **Characteristics**
  - It assumes the pixel intensity changes linearly across the block.
  - Ideal for representing shadows, sky, and soft lighting.



Intra-Prediction: Plane Prediction

- **The Plane Equation**
  - The predicted value $\hat{p}(x, y)$ for a pixel at position (x, y) within the block is calculated using a **planar equation**

$$\hat{p}(x, y) = a + bx + cy$$

    - (x, y): column and row position inside the block
    - a: base intensity
    - b: horizontal gradient (change per column)
    - c: vertical gradient (change per row)
- **Gradient Calculation**

$$b = \frac{Top[N-1] - Top[-1]}{N}$$

$$c = \frac{Left[N-1] - Left[-1]}{N}$$

Top reference

Left reference					
	210	190	170	150	130
220	200	180	160	140	
230	210	190	170	150	
240	220	200	180	160	
250	230	210	190	170	

Predicted 4x4 block

Assume $a = 200,\ b = 10, c = -20$

Plane Prediction: Theoretical Model

Top Reference

210	190	170	150	130

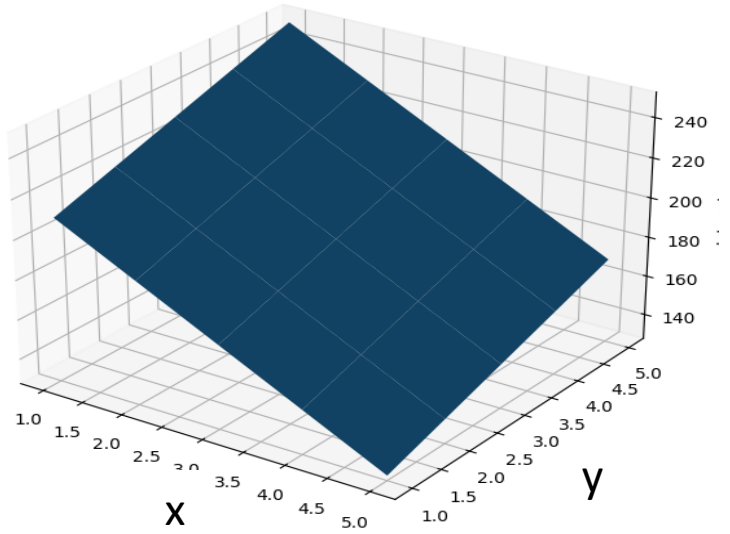
Left Reference

220
230
240
250

$$\hat{p}(x, y) = a + bx + cy$$

Predicted 4x4 block

	x=0	x=1	x=2	x=3
y=0	**200**	**180**	**160**	**140**
y=1	**210**	**190**	**170**	**150**
y=2	**220**	**200**	**180**	**160**
y=3	**230**	**210**	**190**	**170**



Calculating the Gradients

- Horizontal b: (130-210)/4 = **-20**
- Horizontal c : (250-210)/4 = **10**
- Assume a = **200**

- Values decrease by -20 horizontally
- Values increase by 40 vertically
- Form a smooth 2D gradient surface

However, using only end-points to determine the slope is risky.
→ **Least Square formulation**

Plane Prediction : Least Squares Formulation

- **Fit a Plane to a Block**
  - To approximate pixel values using a linear plane: $\hat{x}(i,j) = a + bi + cj$
  - Least Squares Objective

$$\min_{a,b,c} \sum (x(i,j) - (a + bi + cj))^2$$

    - a: base intensity, b: horizontal gradient, c: vertical gradient
  - Parameters are chosen to minimize reconstruction error. → smooth intensity variation
- **Practical Issue**
  - It requires: all block pixels, matrix inversion, float-point computation
  - → Not suitable for real-time video coding

H.264 – From Least Squares to Approximation

- **Key Idea in H.264**: Instead of using all pixels, estimate the plane using only boundary pixels (Top + Left)
- **Simplified Gradient Estimation**
  - The horizontal gradient is estimated from the top reference samples.

$$Top[i] \approx \alpha_T + bi$$

  - The vertical gradient is estimated from the left reference samples.

$$Left[j] \approx \alpha_L + cj$$

  - A least-squares line fitting provides an intuitive interpretation of slope estimation

$$\min_{\alpha_T, b} \sum_i (Top[i] - (\alpha_T + bi))^2 \qquad \min_{L, c} \sum_i (Left[j] - (\alpha_L + cj))^2$$

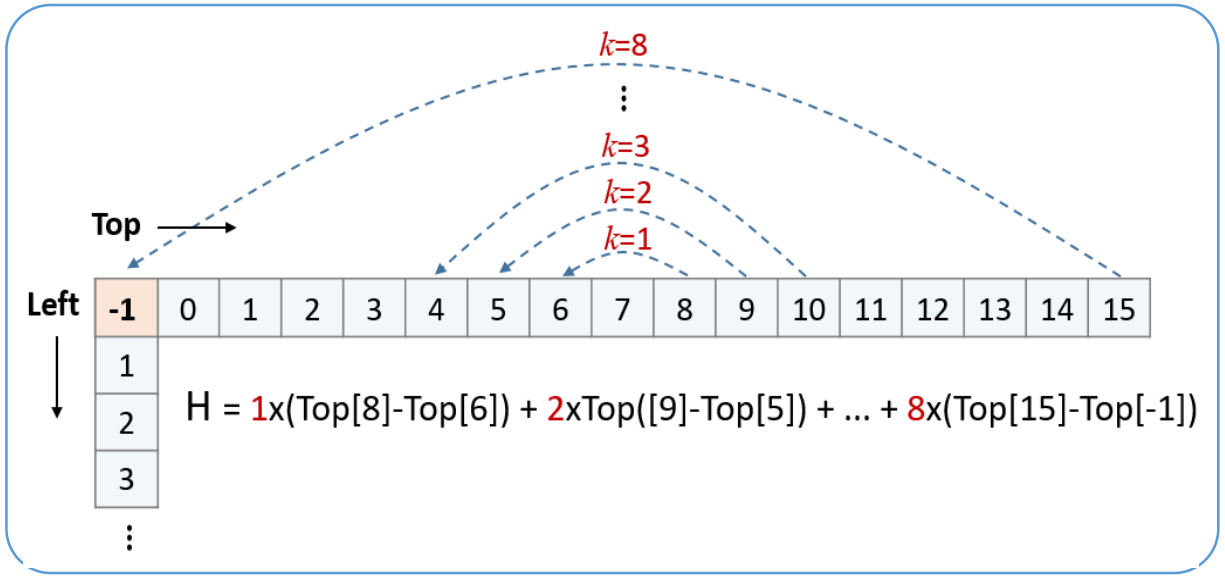
  - **H.264 does not explicitly solve a least-squares problem. Instead, it uses weighted symmetric differences.**

H.264 Plane Prediction

- **H.264 Plane Mode (16x16 blocks)**
  - **Plane equation**

$$\hat{x}(i,j) = \frac{a + b(i-7) + c(j-7) + 16}{32}$$

  - **Gradient calculation**



$$a = 16 \cdot (Top[15] + Left[15])$$ (a is the value of pixel at (7, 7), the overall brightness)

$$b = \frac{5H + 32}{64} \qquad H = \sum_{k=1}^{8} k \cdot (\, Top[7+k] - Top[7-k] \,)$$ (b controls horizontal gradient)

$$c = \frac{5V + 32}{64} \qquad V = \sum_{k=1}^{8} k \cdot (\, Left[7+k] - Left[7-k] \,)$$ (c controls vertical gradient)

H.264 Plane Prediction

- **Advantages of H.264 Plane Prediction**
  - **More robust to local noise**
    - Uses many boundary samples → less sensitive to noise in a single sample.
  - **Better estimate of the overall trend**
    - capture the general horizontal and vertical intensity variation across the block, instead of relying on only two endpoints.
  - **Farther samples contribute more to slope estimation**
    - Samples farther from the center are weighted more heavily
  - **Low computational complexity**
    - Efficient integer implementation using additions and bit shifts.
  - **Good for smooth directional regions**
    - Works well for smooth regions with gradual intensity variation.
  - **Encoder and decoder can produce exactly the same prediction**
    - The integer formulation ensures identical prediction at encoder and decoder.

Plane Mode Summary

- **Why it works?**
  - Natural images often contain smooth illumination changes.
  - Plane prediction uses a linear surface to approximate this change.
  - The residual becomes smaller, leading to better compression.

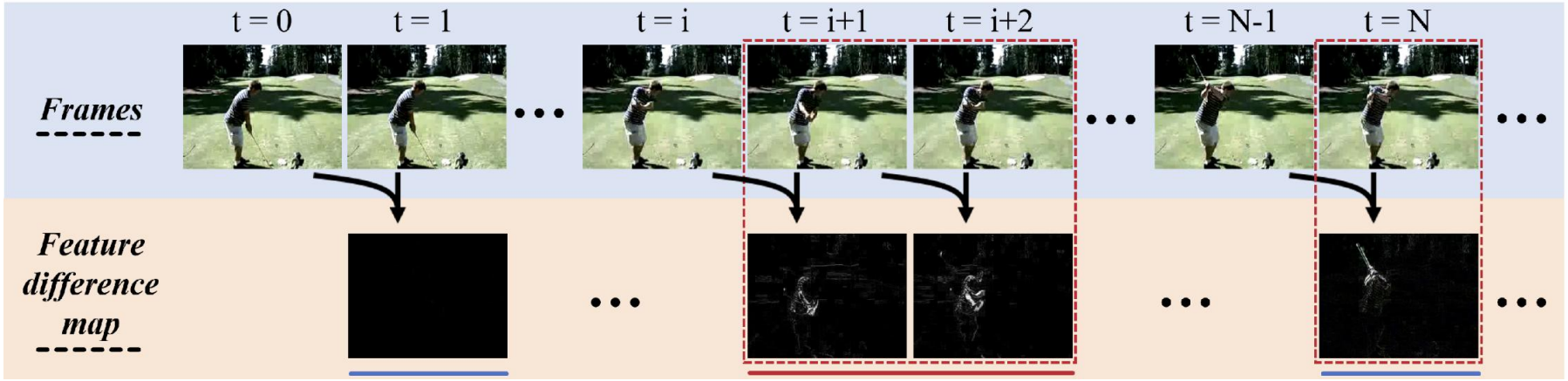
Feature	Theoretical Model	H.264 (Practical)
Data Used	Only the two endpoints.	All 16 boundary pixels.
Slope Calculation	Simple slope between two points.	A "weighted average" slope of the entire boundary.
Robustness	Low: One noisy pixel ruins the whole plane.	High: Noise in one pixel is averaged out by others.
Computation	Floating point	Bit-shifts and Additions only
Standard	Academic / General DSP	H.264 / AVC

Prediction

- The Math of Prediction
- Intra-Prediction
- Inter-Prediction

Inter-Prediction in Video Coding

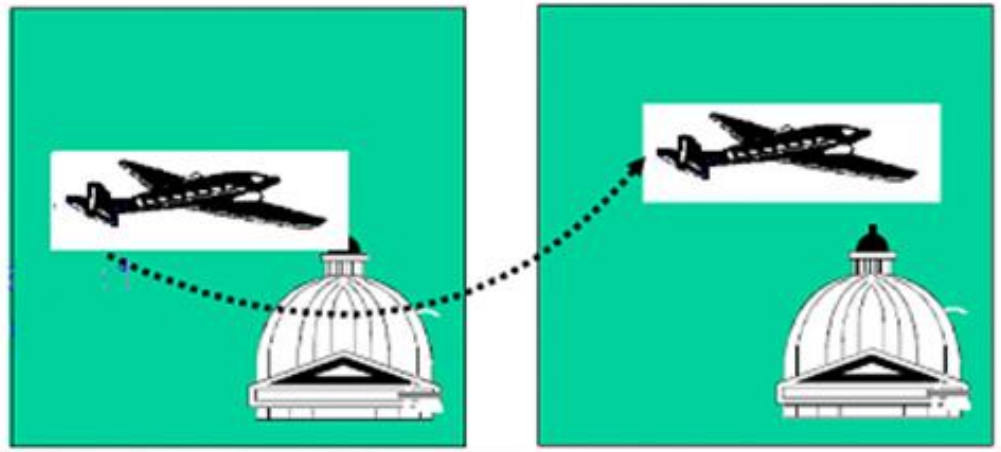
- **Why Inter-Prediction?**
  - **Temporal Redundancy**: Consecutive frames are highly similar
  - Key Idea: Current Frame ≈ Reference Frame + Residual



  - **Compression Ratios**
    - **Intra-prediction:** Typically achieves 10:1 to 20:1.
    - **Inter-prediction:** Can achieve 200:1 or higher.

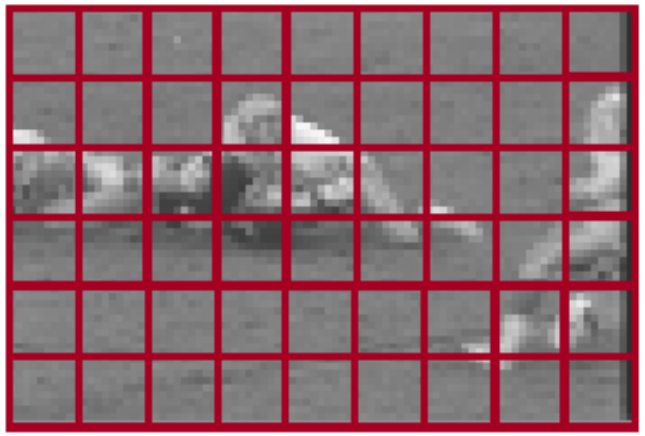
Inter-Prediction

- **Motion Compensation**
  - Consecutive frames are usually very similar, but moving objects may appear at different positions.
  - If we directly subtract two frames, the moving regions may not align.
  - This misalignment creates a large frame difference, even when the object itself has not changed much.
  - Motion compensation tries to align moving content before subtraction.
  - After alignment, the predicted frame becomes closer to the current frame, so the remaining difference becomes smaller and easier to encode..



Inter-Prediction

- **Motion Compensation Unit**
  - **Ideally –** motion for each pixel
    - Too expensive
  - **Semantically** – motion for each homogeneous region or object
    - MPEG-4 Part 2, but too complex
  - **Simplified** – **block-matching** (widely used)
    - Search for an identical or similar block in the reference image
    - Block size
      - MPEG2: 16x16, called macroblock (MB)
      - H.264: 4x4 ~ 32x32



Inter-Prediction

- **Motion Model**
  - **Affine model** – translation, rotation, scaling
    - 6 parameters in 2-D space
    - Complex
  - **Simplified translation model** (used by standard)
    - For each MB in the current frame, all possible displacement in the reference frame is measured.
    - **Motion vectors** (2 parameters)
    - **Coding of motion vectors**
      - A moving object may consist a number of MBs. A greater compression factor is obtained if the MVs are transmitted *differentially.* Since adjacent MBs may have the same vectors, the vector difference becomes zero.

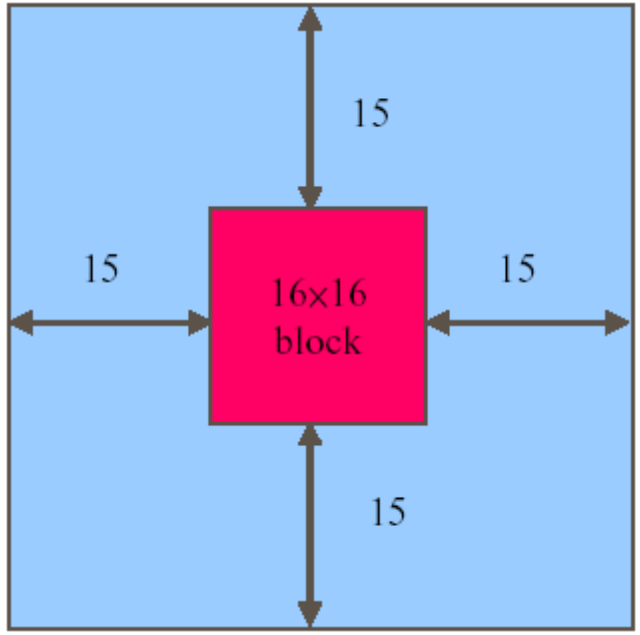
Inter-Prediction

- **Block Matching Algorithm (BMA)**
  - Definition: Divide the current frame into blocks (e.g., 16x16) and shift them within a "**Search Window**" in the reference frame.
  - Goal: Look for one with minimum MSE or SAD
- **Matching Criteria**
  - SAD (Sum of Absolute Differences):

$$\sum |p_i - q_i| \quad \text{(Hardware-friendly, fastest)}$$

  - SSE (Sum of Squared Errors):

$$\sum (p_i - q_i)^2 \quad \text{(Mathematically related to PSNR)}$$



Motion Estimation (ME) is the most computationally intensive part of the encoder.

Inter-Prediction

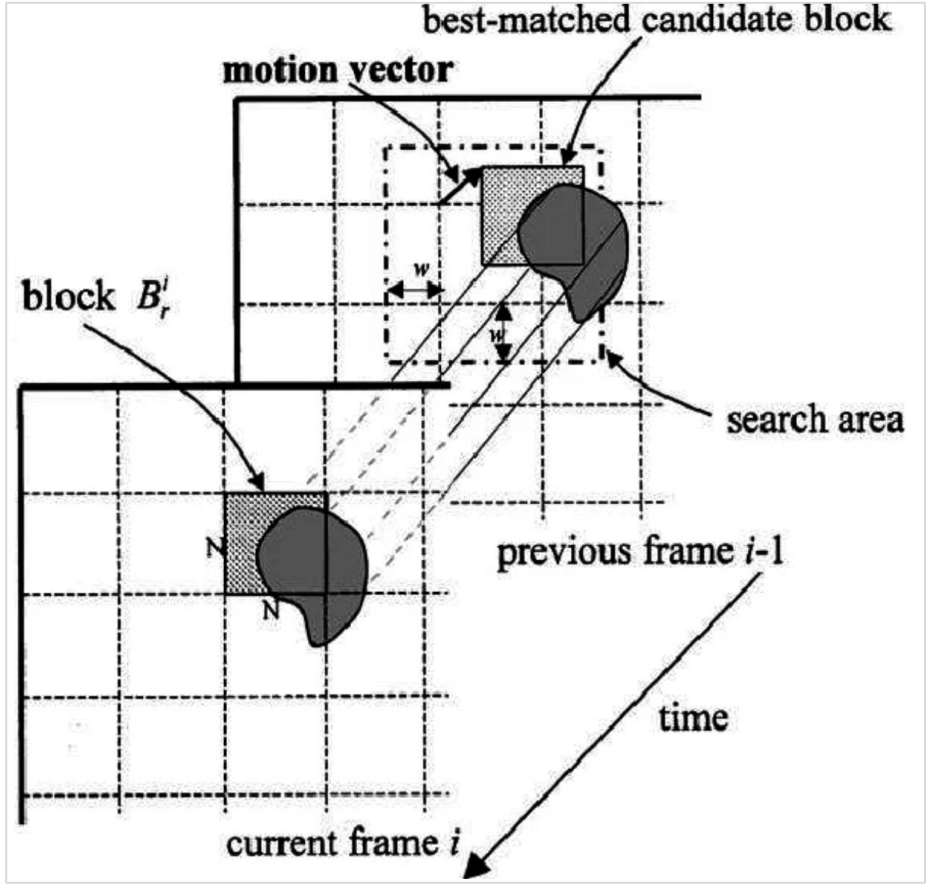
- **Encoding Pipeline**
  - **Motion Estimation (ME)**: Find similar block in reference frame
  - Compute **motion vector (MV)**: mv_x, mv_y
  - **Motion Compensation (MC)**

$$\hat{B}(x,y) = B_{ref}(x + mv_x, y + mv_y)$$

  - Encode MV , residual $R = B - \hat{B}$
- **Decoding Pipeline**
  - Decode: get MV, residual $\tilde{R}$
  - Motion Compensation (MC)

$$\hat{B}(x,y) = B_{ref}(x + mv_x, y + mv_y)$$

  - Reconstruction: $\tilde{B} = \hat{B} + \tilde{R}$



Inter-Prediction - Motion Estimation (ME)

- **Search Strategies**
  - **Full Search (FS):** Checks every single pixel within the search window.
    - Pros: Optimal solution (Guarantee to find the global minimum SAD)
    - Cons: Computationally expensive.
  - **Fast Search Algorithms:** Reduce candidate blocks in the search window.
    - Pros: Low computational complexity
    - Cons: Local minimum SAD
    - E.g., Three-Step search / Diamond search / Hexagon search

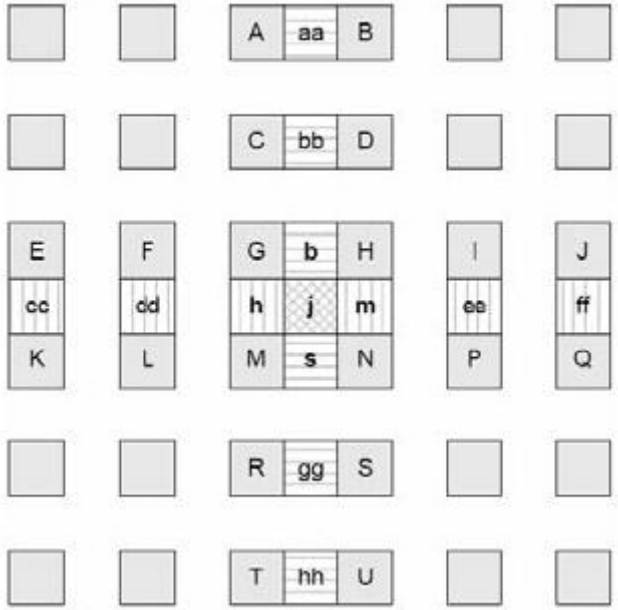
Compromise between efficiently removing temporal redundancy and computational cost.

Inter-Prediction - Sub-pixel Motion Estimation

- **Motivation for Sub-Pixel ME**
  - Real-world motion is continuous, not quantized to pixels. Using integer-pel precision has less precision → more residual
- **Solution:** Using fractional-pel (1/2, 1/4 pixel) to improve accuracy
  - **Half-pel** (1/2 pixel)**:**
    - linear interpolation (MPEG2), 6-tap filter (H.264), 8-tap filters (HEVC)
  - **Quarter-pel** (1/4 pixel) **:**
    - linear interpolation
  - **Pros**: Fractional-pixel precision can offer a ~2dB gain over integer precision
  - **Cons**: Increases search points and significantly raises memory bandwidth requirements.

Sub-pixel ME/MC

6-tap FIR (1/2-pixel)



6-tap FIR: (1, -5, 20, 20, -5, 1)/32

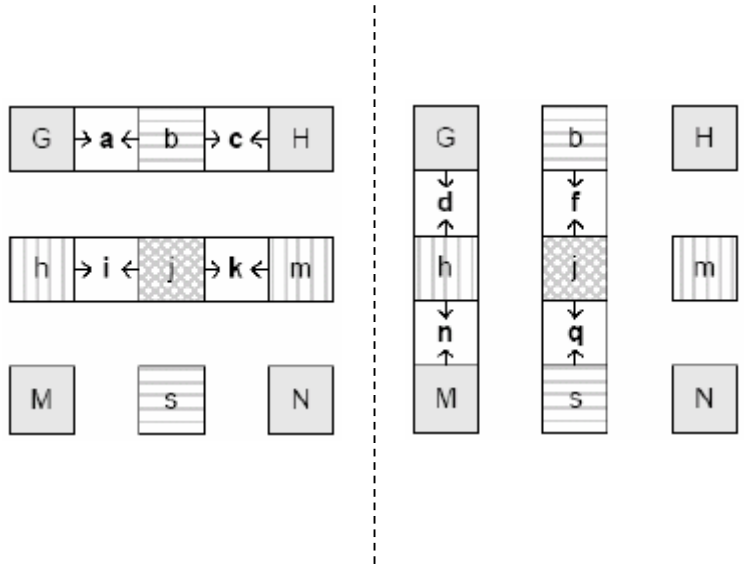
b = round((E-5F+20G+20H-5I+J)/32)

h = round((A-5C+20G+20M-5R+T)/32)

j = round((cc-5dd+20h+20m-5ee+ff)/32)

or round((aa-5bb+20b+20s-5gg+hh)/32)

Bilinear interpolation (1/4-pixel)



a = round((G+b)/2)

d = round((G+h)/2)

MV coding and Residual Coding

- **Motion Vector Coding**
  - **The Problem:** Transmitting raw for every block consumes too many bits.
  - **Observation:** Motion exhibits high "Spatial Correlation"—neighboring blocks usually move in the same direction.
  - **Differential Coding:** Only transmit the **MVD** (Motion Vector Difference)
    - MVD = $MV_{actual} - \boldsymbol{MVP}$ (MVP: predicted MV)
    - Different compression standards define their MVP selection
    - MVD uses lossless coding
- **Residual Coding**
    - Transform, Quantization (so, it is lossy coding), Entropy Coding
    - Residual has very low energy → high coding efficiency

Advantage Topics

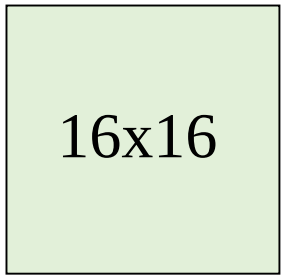
- **Block Partitioning**
- **Motion Estimation**
- **Motion Information Coding (MV coding)**

Block Partitioning

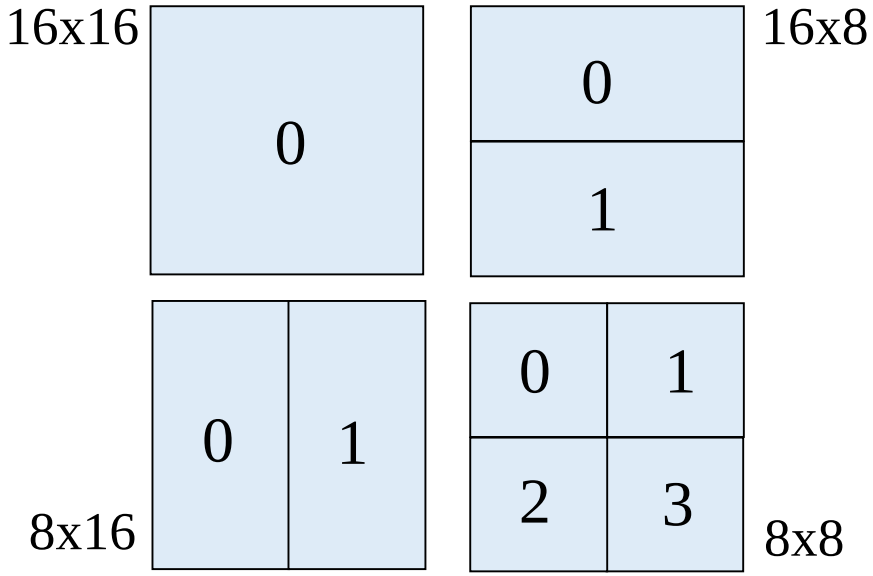
- **The Problem:** Natural images are not uniform (smooth region, edges, textues)
  - Large fixed blocks cannot model details → blur details in complex areas.
  - Small fixed blocks waste too many bits (MVs, modes) in flat areas.
  - → Using a fixed block size (e.g., 16x16 in MPEG2) is inefficient
- **Solution:** Use adaptive block sizes to match content
- **Evolution:**
  - MPEG2: Macroblocks (MB): 16x16
  - H.264/AVC: Flat selection from 7 modes (up to 16x16)
  - HEVC: CTUs (up to 64x64), using Quad-tree with recursive splitting
  - VVC: CTUs (up to 128x128) with Multi-type Tree

Block Partitioning

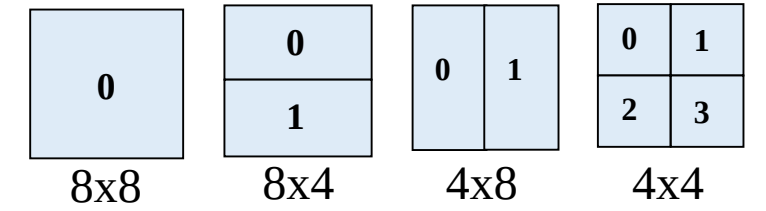
- **MPEG2: Macroblocks (MB): 16x16**



- **H.264/AVC: Flat selection from 7 modes (16x16 to 4x4)**

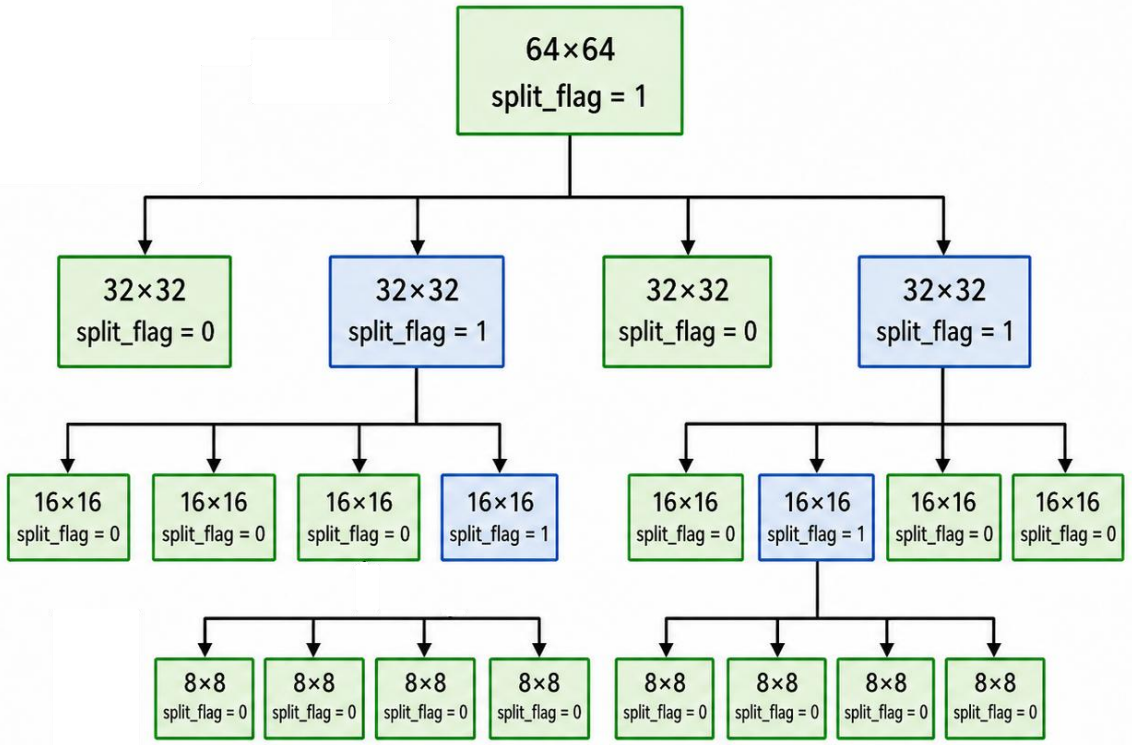


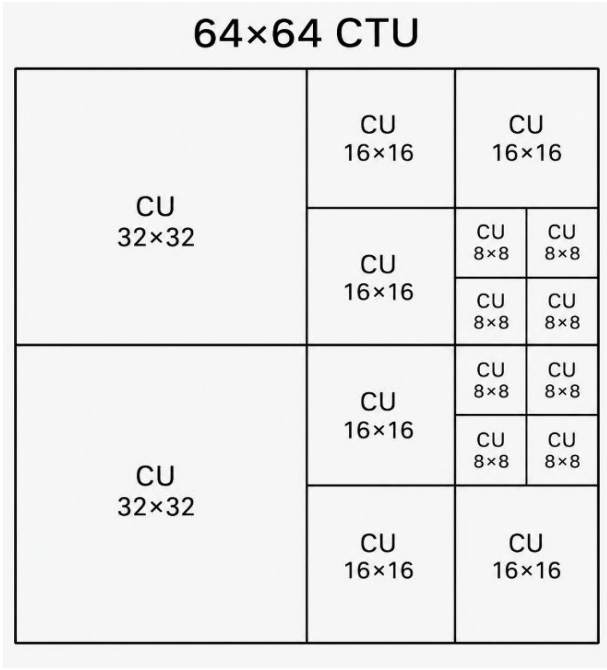
These sub-partitions can be used only when the macroblock is first partitioned into 8x8 blocks.



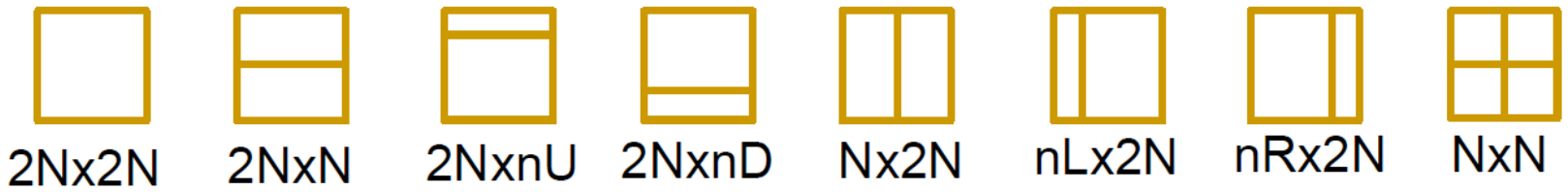
Block Partitioning

- **Quad-tree with Recursive Splitting** (up to 64x64)





Inter-CU:

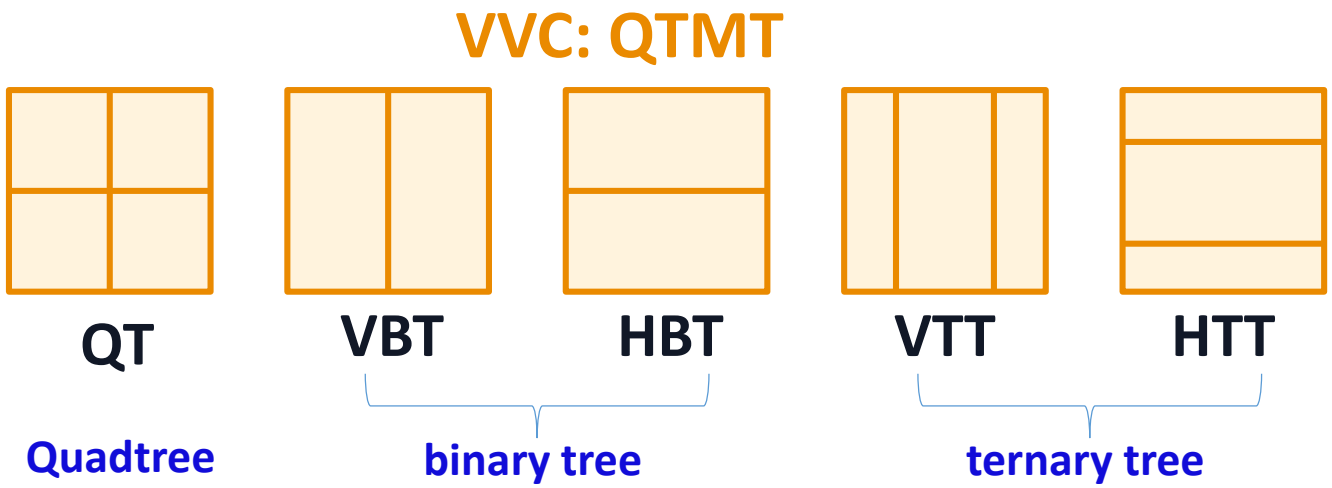


Block Partitioning

- **Quad-tree with Recursive Splitting**
  - RDO-Based Recursive Search
    - The encoder performs a **top-down search** for all possible CU partitions.
    - For each CU candidate, it tests all available **Intra** and **Inter** prediction modes
  - The Decision Rule
    - It compares the **Rate-Distortion Cost** of the parent CU against the sum of its four children CUs.
    - The configuration with the **lowest cost** is selected and written into the bitstream.
  - Why it's slow:
    - This "exhaustive search" is the reason why HEVC encoding takes much more CPU power than H.264. Modern encoders often use **Fast RDO** (based on AI or heuristics) to skip unlikely split paths.

Block Partitioning

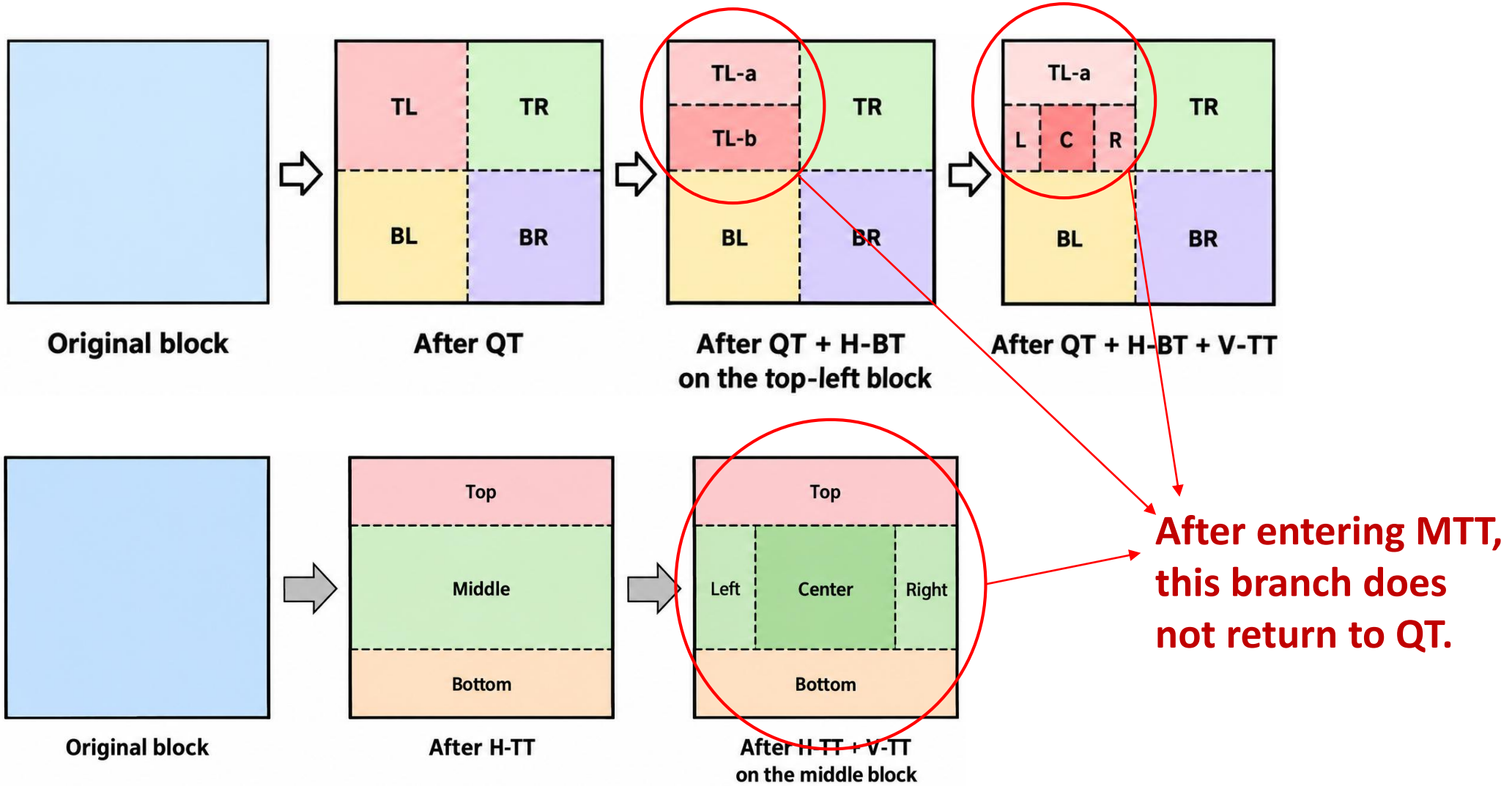
- **Multi-Type Tree** (up to 128x128)
  - **Quad-tree Limitation**: Only supports square partitioning at the CU level, which struggles with diagonal edges or horizontal/vertical motion. → Solution: QTMT
  - **QTMT: Quadtree + binary tree (1:1 ratio) + ternary tree (1:2:1 ratio)**
  - Horizontal and vertical splits create rectangular CUs
  - It allows the partition shape to better match local image structures.
  - More partition choices improve coding efficiency, but they also greatly increase the number of candidate partition trees that the encoder must evaluate.



Block Partitioning

- **QTMT Pipeline:** The encoder evaluates the CTU through a nested search
  - Step A: **Quad-tree (QT) Phase**
    - A block may be split into four equal square sub-blocks.
    - QT splitting can be applied recursively, subject to size/depth constraints.
    - At any node, the encoder may stop QT splitting and enter the MTT stage.
  - Step B: **Multi-type Tree (MTT) Phase**
    - Once a block enters the MTT stage, it may choose one of four split types:
      - **Horizontal Binary (H-BT)** (Split 1:1)
      - **Vertical Binary (V-BT)** (Split 1:1)
      - **Horizontal Ternary (H-TT)** (Split 1:2:1)
      - **Vertical Ternary (V-TT)** (Split 1:2:1)
    - After an MTT split, further splits may still use BT or TT. The partitioning process cannot return to QT splitting below that branch.

MTT: Example


TL
TR
BL
BR
TL-a
TL-b
L
C
R
Top
Middle
Bottom
Left
Center
Right
Original block
After QT
After QT + H-BT
on the top-left block
After QT + H-BT + V-TT
Original block
After H-TT
After H-TT + V-TT
on the middle block
After entering MTT,
this branch does
not return to QT.

Advantage Topics

- Block Partitioning
- Motion Estimation
- Motion Information Coding (MV coding)

Motion Estimation (ME)

- **Search Strategies**
  - **Full Search (FS):** Checks every single pixel within the search window.
  - **Fast Search Algorithms:**
    - Reduce candidate blocks in the search window.
    - Compromise between efficiently removing temporal redundancy and computational cost.
    - Examples:

      1. Three-Step search (TSS)

      2. Diamond search (DS)

      3. Hexagon search (HS)

      4. UMHexagonS (used in H.264/AVC)

      5. TZ Search (used in H.265/HEVC)

Motion Estimation – Three-Step Search (TSS)

- **Core Concept**
  - Coarse-to-fine search
  - Iteratively reduce step size
- **Strategy**

```
Initialize center = (0,0)
Set step size s

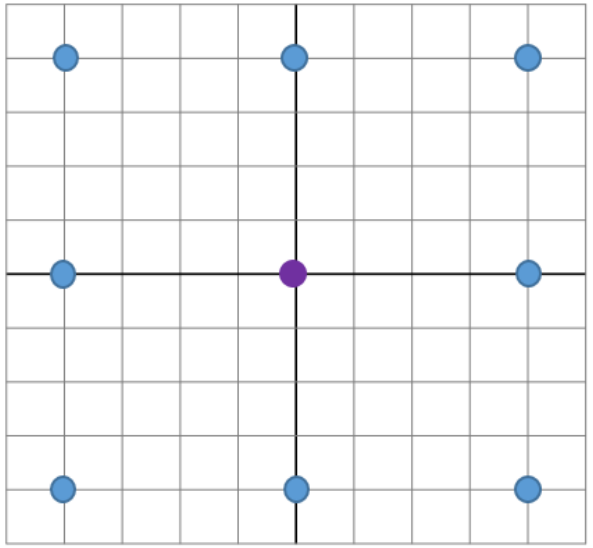
Repeat:
    Evaluate 9 points using step size s
    Move center to best point
    s = s / 2
Until s = 1

Return best match
```

Search Pattern

- Center: (x, y)
- 8 neighbors: $(x \pm s, y)$
  $(x, y \pm s)$
  $(x \pm s, y \pm s)$

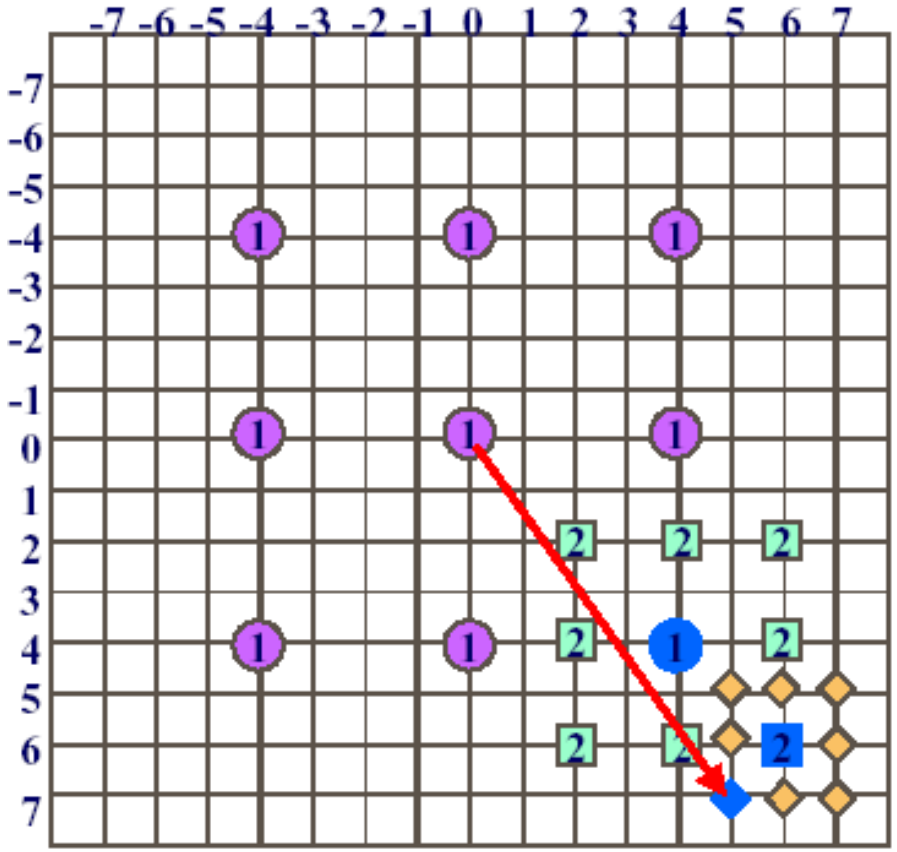
(e.g., s = 4)



Motion Estimation – TSS

- **Three Step Search**
  - Initial step size s = 4
  - Three steps with size s:

    4 → 2 → 1
  - Search window: ±7 (7=4+2+1)
  - Total: 9+8+8 = **25** block matching

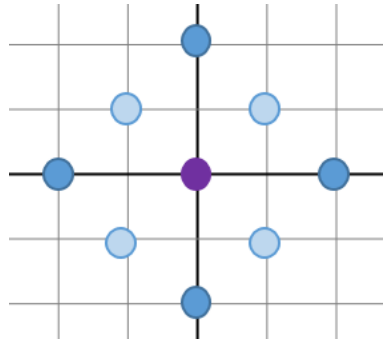


- **Comparison with Full Search**
  - Search range: ±7 → 15x15 candidate positions
  - **Full search:** 15x15= **225** block matching per MB

Motion Estimation – Diamond Search (DS)

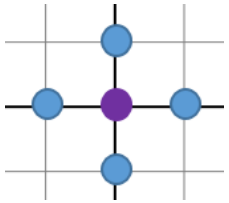
- **Core Concept**
  - Use **Large Diamond (LDSP)** for exploration
  - Use **Small Diamond (SDSP)** for refinement
- **Algorithm**

```
Initialize center = (0,0)
Repeat:
    Apply LDSP
    Find best point
    If best == center:
        break
    Else:
        Move center to best point
Apply SDSP
Return best match
```



LDSP Structure (9 points)

- Center : (x, y)
- 4 axial points (distance = 2) : $(x\pm2, y), (x, y\pm2)$
- 4 diagonal points (distance = $\sqrt{2}$): $(x\pm1, y\pm1)$

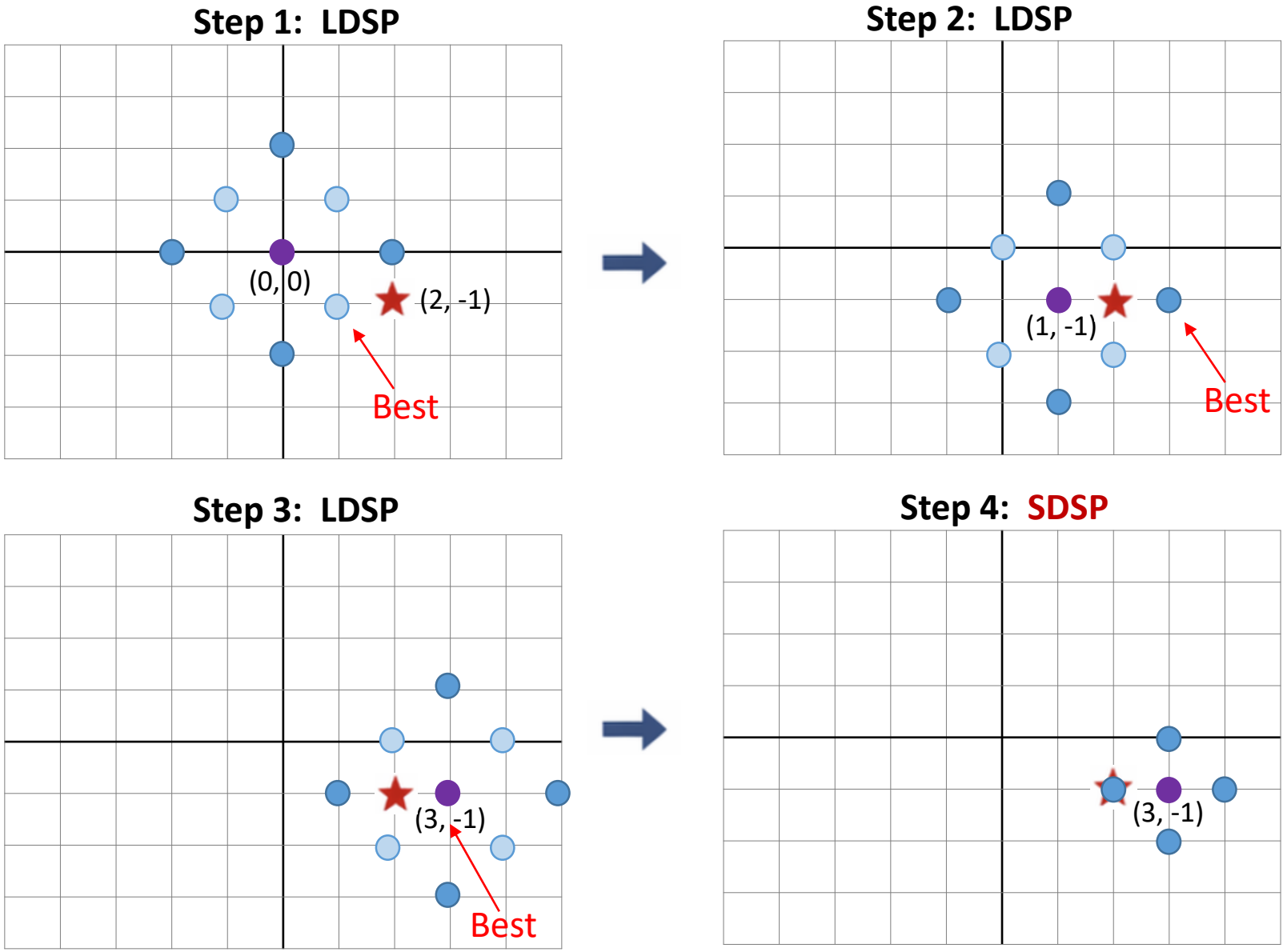


SDSP Structure (5 points)

- Center (x, y)
- 4 axial points (distance = 1) :$(x\pm1, y), (x, y\pm1)$

Motion Estimation

Diamond Search - Example



Motion Estimation – Diamond Search (DS)

- Diamond Search is a fast block-matching method for motion estimation.
- It uses a diamond-shaped search pattern and moves the search center adaptively toward the best matching position.
- The search continues with the Large Diamond Search Pattern (LDSP) until the best match is at the center, then switches to the Small Diamond Search Pattern (SDSP) for final refinement.
- In practice, the search is still limited by a predefined maximum search window.
- More adaptive than Three-Step Search - Unlike Three-Step Search, it does not follow a fixed step-size schedule.
- Often converges quickly when the true motion vector is near the current search center - Does not waste comparisons on large fixed steps when the motion is small.

Motion Estimation – Hexagon Search

- **Core Concept**
  - Use hexagon-shaped search pattern. (6 + 1 points)
  - Cover wider area than diamond, but use fewer points.
- **Algorithm**

Initialize center = (0,0)

Repeat:

Apply **Large Hexagon Pattern (LHP)**

Find best point

If best == center:

break

Else:

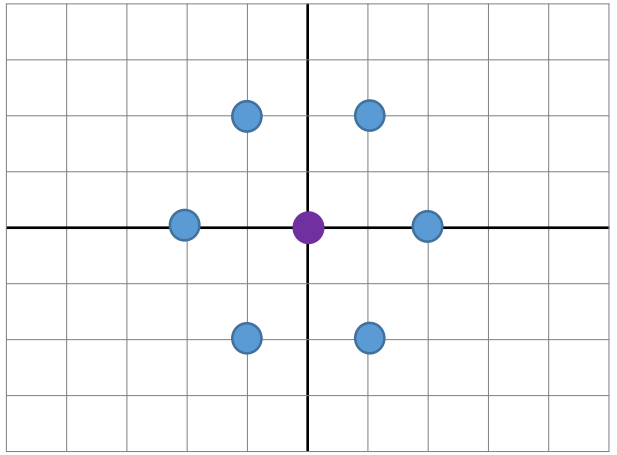
Move center to best point

Apply **Small Diamond Search Pattern (SDSP)**

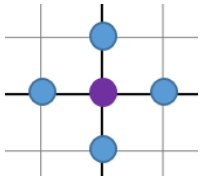
Return best match

LHP Pattern

- Center: (x, y)
- 6 points: ($x \pm 2, y$), ($x \pm 1, y \pm 2$)



SDSP Pattern



- Center: (0, 0)
- 4 points: ($x \pm 1, y$), ($x, y \pm 1$)

Comparison – TSS, DS, HS

- **TSS:** Square grid (9 points)
  - Coarse → fine (global → local) by step-wise reduction (e.g., s = 4, 2, 1)
  - Typical search points: 25~30 (medium efficiency)
  - Hardware control logic: very simple (hardware friendly)
- **DS:** Diamond (LDSP + SDSP)
  - Gradient-like local search
  - Typical search points: 13~17 (high efficiency)
  - Hardware control logic: moderate
- **HS:** Hexagon + small diamond (SDSP)
  - Faster directional convergence
  - Typical search points: 11~15 (highest efficiency)
  - Hardware control logic: slightly complex (UMHexagonS used in H.264 software)

Motion Estimation – TZ Search

- A fast motion-estimation search algorithm designed to reduce the complexity of Full Search.
- Combines multiple search strategies rather than relying on a single fixed pattern.
- Uses motion-vector predictors to start from promising locations.
- Performs multi-scale grid search for efficient coarse exploration.
- Uses Raster Search only when necessary to recover from a poor initial region.
- Finishes with local refinement around the current best motion vector.
- Balances: search accuracy, computational complexity, robustness against local minima
- TZ Search is a hierarchical and adaptive search framework that combines prediction, coarse multi-scale search, optional wide-range raster search, and local refinement.

Motion Estimation – TZ Search

The 4-Stage Workflow

- **Stage 1: Initial Predictor Selection**
  - Select initial search center (e.g., MV of top, left, and collocated blocks).
- **Stage 2: Initial Search**
  - Perform a diamond/square search around the initial center using increasing search distances.
  - Find a promising motion vector and decide whether a wider search is needed.
- **Stage 3: Raster Scan (The "Test Zone"):**
  - If the Stage 2 result is still far from the initial center, perform a sparse-grid search over the search window to locate a better candidate region.
- **Stage 4: Refinement**
  - Start from the current BestMV and perform local diamond/square refinement. If a better point is found, recenter and repeat until no further improvement is obtained.

Motion Estimation – TZ Search

- **Stage 1: Initial Predictor Selector**
  - Use motion vector predictors (MVPs):
    - spatial neighbors (left, top, etc.)
    - temporal predictor
    - zero MV
  - Evaluate initial costs and select best candidate(s)
- **Stage 2: Initial Search**
  - Perform a diamond or square grid search around the initial center.
  - Test search patterns with increasing strides:

    1, 2, 4, 8, … up to the allowed search range
  - Each pattern typically evaluates 8 points
  - Update best MV whenever a better point is found
  - Record the stride of the best point as BestDistance

Decision after stage2

If BestDistance = 0 → terminate

If BestDistance > R → Stage 3: Raster Search

if 0 < BestDistance < R → Stage 4: Refinement

Motion Estimation – TZ Search

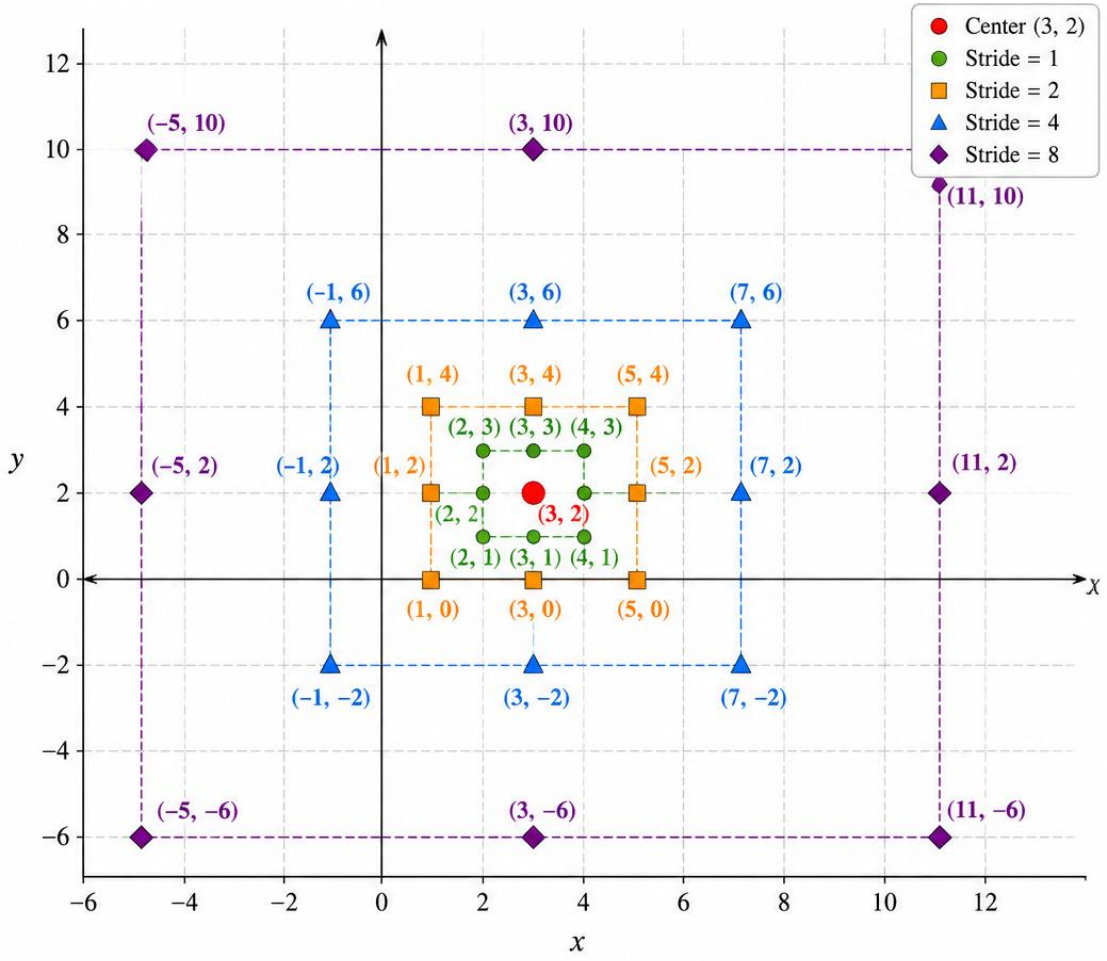
- **Stage 3: Raster Search**
  - Performs a **sub-sampled full search** over a large search window.
  - Instead of checking every displacement, it evaluates candidates on a regular sparse grid. (e.g., with raster spacing R)

Early termination: If the cost at any grid point falls below a dynamic threshold, the search terminates immediately, moving directly to refinement stage.

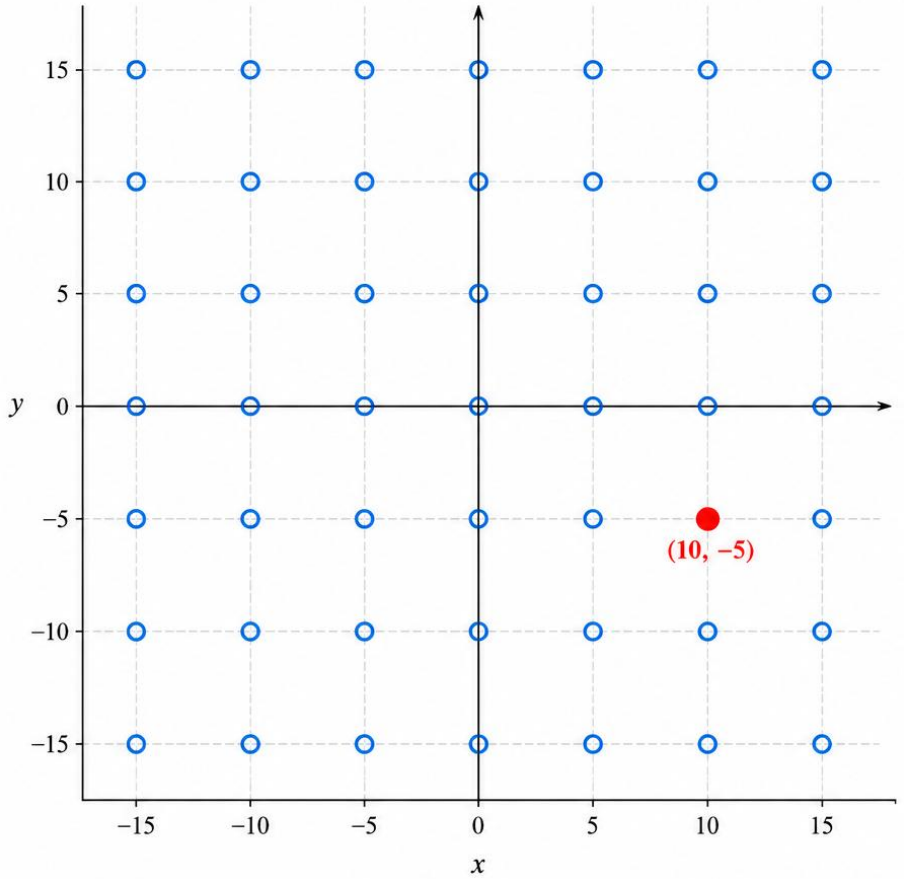
- **Stage 4: Refinement**
  - Start from the current **BestMV** as new center.
  - Test diamond/square patterns at distances: 1,2, 4, 8, ...
  - Keep the best candidate found during the whole refinement round.
  - If a better point is found, recenter at the new BestMV and repeat.
  - Stop when the center remains the best point.

TZ Search - Example

Stage 2: Initial Search



Stage 3: Raster Search



Search window: x, y in [-5, 5]
Search with spacing R = 5
Evaluate all (x, y) in the grid (7x7=49 points)

Motion Estimation Summary

- **TSS**: a pathfinder (開路先鋒)
  - It rapidly explores the search space using a coarse-to-fine strategy
  - Simple, predictable, and computationally efficient.
- **Diamond Search (DS)** and **Hexagon Search (HS):** scouts (偵察兵)
  - Adaptively move the search pattern toward better matching positions.
  - DS explores reactively while HS moves more directionally with a larger step size.
- **TZ Search:** an intelligence officer (情報分析官)
  - Uses available motion information, such as spatial and temporal predictors, to choose promising starting points.
  - Combines **multi-scale search, optional raster search, and local refinement**.
  - Avoids unnecessary wide-range search when a good MV can be found quickly.
  - Used in H.265/HEVC ref. software (HM)

Advantage Topics

- Block Partitioning
- Motion Estimation
- Motion Information Coding (MV coding)

Advanced Techniques - AMVP

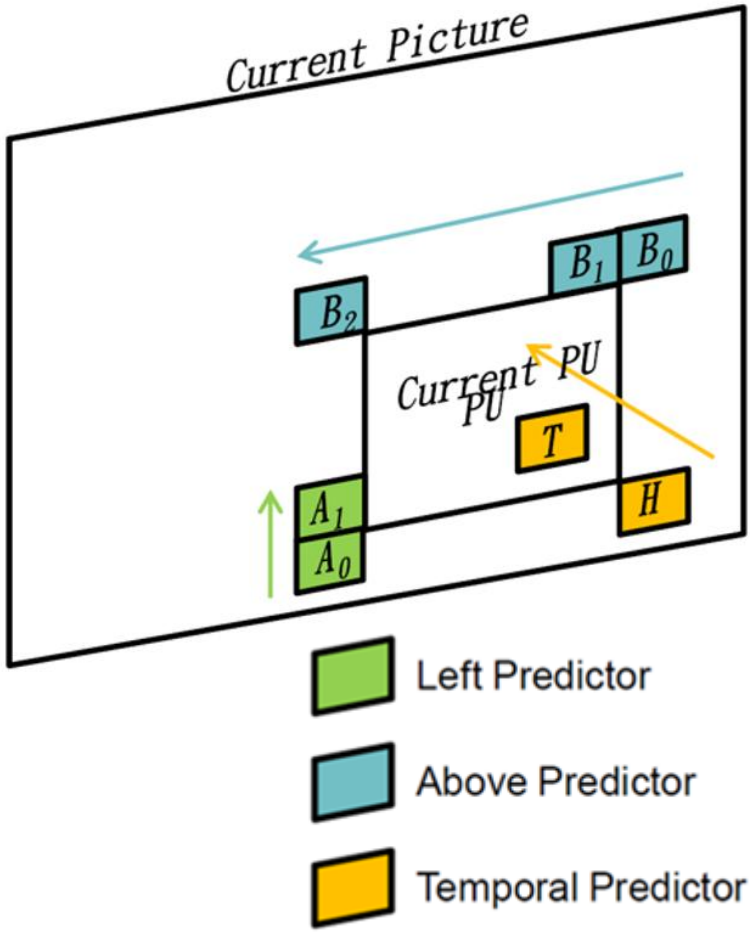
- **What is AMVP** (Advanced Motion Vector Prediction)
  - A technical framework used in modern video compression to reduce the bit rate required to transmit Motion Vectors (MV)
  - Motivation: 1. Encoding raw MVs for every block costs too much bits
    2. Neighboring blocks usually have similar motion
  - Idea: Predicts MV from neighbors, and only encode difference (MVD).

$$\boldsymbol{MV = MVP + MVD}$$ (MVP: motion vector predictor)

- **AMVP Framework**
  - Encoder : 1. Generate candidate MVP list
    2. Encoder selects best predictor
    3. Encode: MVP index, MVD
  - Decoder : 1. Reconstruct MVP list
    2. Use index → MVP, and then MVP + MVD → recover MV

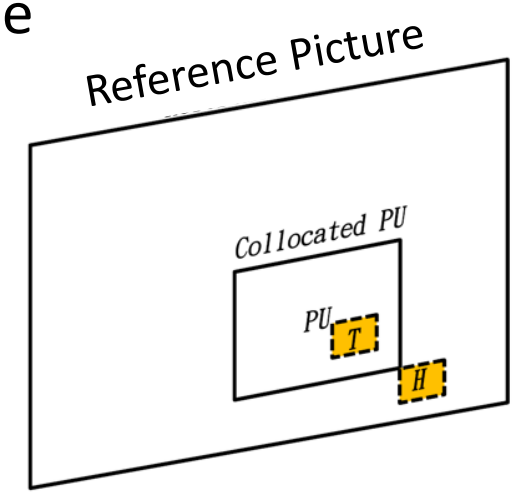
AMVP

- **Spatial MV Candidates**
  - Exploiting correlation between a current block and its spatial neighbors.
  - **Candidate Groups**
    - **Left Group**: bottom-left (A0) and left (A1)
    - **Above Group**: top-right (B0), above (B1), and top-left (B2)
  - **Pruning (Deduplication)**: Duplicated candidates will be removed to ensure the list offering truly different choices, maximizing coding efficiency.



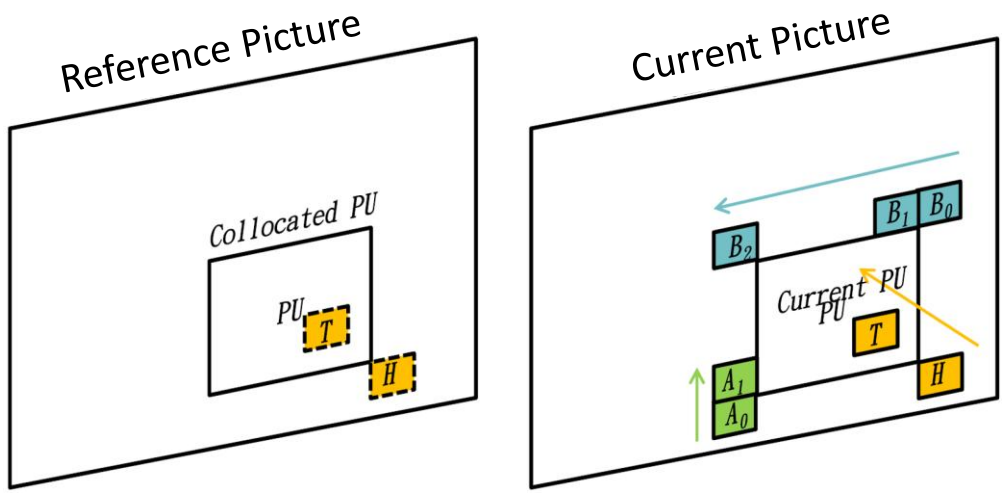
AMVP

- **Temporal MV Candidates**
  - Exploiting motion consistency across different frames in time.
  - It looks at the MV of a block in a previously encoded "Reference Frame" at the same relative coordinates.
  - **Search Positions**
    - Primary (H): Outside the bottom-right corner
    - Secondary (T): The center of the co-located block



  - **MV Scaling:** Since the time distance between frames varies, the temporal MV is scaled to match the current frame's timing.
  - What is actually sent in the Bitstream? ***MVP, MVD, reference index***

AMVP



- **Candidate list construction**
  - **Search Spatial Candidates**
    - Left (A0, A1): Search Order: A0 → A1
      - If A0 is available, A1 will not be checked.
    - Above (B0, B1, B2): Search Order: B0 → B1 → B2
      - The first available MV found is added to the list, then the search stops.
    - **Pruning**: If the two candidates in the list are identical, one is removed.
  - **Search Temporal Candidates**
    - Searched only when the list is not full (AMVP limits the List to 2 entries)
    - Temporal : Search order: H → T
      - If H is available, then the search stops (T won't be checked)
    - **Scaling:** If the time distance between frames is different, the MV is scaled.
- **Padding**: If the list is still not full after spatial and temporal checks, Zero MVs (0,0) are added.

AMVP

- **Encoder Decision – Rate-Distortion Optimization (RDO)**
  - **The Competition:** The Encoder tests every candidate in the final list.
    - For each candidate i, the encoder calculates:

$$J = \text{Distortion} + \lambda \times \text{Rate}(\text{Index}_i, \text{MVD}_i)$$

  - **Final Output:** The encoder chooses the index that results in the lowest "Cost" (best balance of quality vs. bits) and writes it into the bit-stream.
- **Why not more neighbors?**
  - **Overhead:** A list of 2 needs only 1 bit for the index. Larger lists increase the bit-cost.
  - **Pruning Complexity:** Every candidate must be compared against existing ones to ensure uniqueness. More candidates = more hardware latency.
  - **Diminishing Returns:** The immediate neighbors (A1, B1) are statistically the most accurate. Adding distant neighbors rarely improves prediction enough to justify the bit-cost.

Advanced Techniques

After AMVP, what's next?

Improve prediction, representation, and motion modeling

1. **HMVP** (History-based MVP)
   - Reuses motion vectors from previously coded blocks as additional prediction candidates
2. **AMVR** (Adaptive MV Resolution)
   - Adapts MV precision to achieve a better rate–distortion trade-off
3. **SMVD** (Symmetric MVD)
   - Encodes a single MVD and derives the other using a symmetry constraint.
4. **Affine AMVP**
   - Models non-linear motion using control-point-based motion vector prediction

HMVP (History-based MVP)

- **Motivation:** Traditional AMVP uses spatial and temporal neighbors to predict motion vectors (MVs). However, these candidates are limited to local information.
- **HMVP**
  - Extends the MVP candidate pool by reusing motion information from previously coded blocks. It maintains a history buffer of MV candidates.
  - Uses a small FIFO buffer to store MVs, collected from previously coded blocks
  - For the current block, it combines:
    - Spatial MVPs
    - Temporal MVPs
    - **History-based MVPs**

    } Use RDO to select the best MVP
  - HMVP enhances motion prediction by reusing recent motion patterns within the same frame, providing non-local yet compact and efficient predictors.

AMVR (Adaptive MV Resolution)

- **Why AMVR**
  - In video coding, MVs are typically represented with high precision (1/4-pixel).
  - However: High precision → better prediction accuracy

    High precision → larger bit cost (more bits for MVD)
  - Not all blocks require high precision
    - Smooth or large-motion regions: coarse MV is sufficient → fewer bits
    - Complex or detailed motion: fine MV improves accuracy
- **AMVR** (Adaptive Motion Vector Resolution)
  - Adaptively adjusts the precision (resolution) of motion vectors per block.
  - For each block, the encoder selects an MV resolution level based on RDO
  - The chosen MV resolution is signaled (implicitly or explicitly) to the decoder.
  - Widely used in modern codecs (HEVC extensions, VVC)

SMVD (Symmetric Motion Vector Difference)

- **Motivation**
  - In bi-prediction (Bi-pred), a block uses **two motion vectors**:
    - one from list 0 (forward reference)
    - one from list 1 (forward reference)
  - Traditionally, both MVs are coded independently → high bit cost
- **SMVD** (Symmetric Motion Vector Difference)
  - Assume that the motion with respect to two reference frames is symmetric.
    - MVD = - MVD so, → $MV_0 = MVP_0$ + MVD
    - $MV_1 = MVP_1$ - MVD (this MVD is derived implicitly)
  - Exploit temporal symmetry to reduce redundancy in motion representation .
  - Encoder uses RDO to decide whether to enable SMVD

SMVD

- **What happens in SMVD mode?**
  - The encoder still generates: MVP_0, MVP_1 (using AMVP/HMVP/…
  - However, during motion estimation, only **one parameter MVD** is optimized.
    - the two motion vectors are coupled with a symmetric constraint
  - The resulting constrained motion vectors are used for prediction

$$J = D + \lambda R$$

  - and compared with the cost of conventional bi-prediction

Even in the case of linear motion, MVP_0 and MVP_1 are generally not identical, and thus the resulting MVDs are unlikely to be perfectly symmetric. However, as long as the motion is approximately symmetric, SMVD can still achieve a better rate–distortion trade-off.

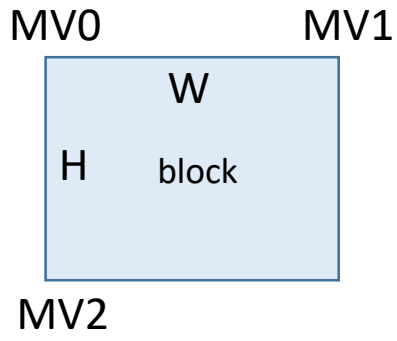
	Distortion	Rate
Normal	↓（better）	↑（2 MVDs）
SMVD	↑（worse）	↓↓↓（1 MVD）

Affine Motion

- **Why Affine Motion:** Traditional motion compensation assumes only "Translation" (shifting) motion, where all pixels in a block share the same MVs.
- **Affine Motion (**Non-linear Motion**)**
  - Support non-linear motions: **Rotation**, **Zooming**, and **Shearing**
  - Instead of a single MV, it uses multiple **control points** to describe motion field for each pixel in the block

$$MV(x,y) = MV_0 + \frac{x}{W}(MV_1 - MV_0) + \frac{y}{H}(MV_2 - MV_0)$$

MV_0 , MV_1 , MV_2 : control point motion vectors, W, H: block width and height



  - **4-parameter affine:** 2 control points (top-left (MV_0), top-right (MV_1))
  - **6-parameter affine**: 3 control points (top-left, top-right, bottom-left (MV_2))

- **Affine AMVP:** $MV = MVP + MVD$

MVP = predicted control-point MVs
MVD: difference for each control point

Summary & Future Outlook

- Inter prediction exploits temporal redundancy
- Core = Motion Estimation + Compensation
- Residual is small → efficient compression
- Modern codecs improve
  - Motion modeling, Block partitioning, Prediction modes
  - Inter-prediction has evolved from simple block matching to complex geometric and optical models.
- AI Integration
  - Neural-network-based motion compensation (NNMC) is the next frontier for future standards.